

Written exam 17 December, 2024

Course title: Network Optimization

Course no: 42115

Aids allowed: All

Exam duration: 4 hours

Weighting: The points are indicated at each question

Language: You can write in Danish or English

Figures: Figures can be found on the last pages of this assignment. You can use them in your answer. Pull out the individual pages, and insert them in your answer.

Critical path with time windows



Old-fashioned Danish apple cake (gammeldags æblekage) is a classic Danish dessert. Although it is called apple cake, it has more in common with a trifle or a crumble. It is a lovely dessert on top of a good meal, and then it is pretty easy to make. Æblekage is always served cold with a generous layer of whipped cream. Making the apple cake, consists of the below six steps $A, \dots, F$.

Now, assume that each task j has a corresponding time windows $[e_j, \ell_j]$ denoting the earliest and latest time when the task may start.

activity i	description	duration d_i	predecessor	e_j	ℓ_j
A	Boil apples	2	B, D	0	∞
B	Peel apples and cut into cubes	6		0	∞
C	whip cream	9	D, E	0	6
D	slice vanilla bean, scrape out seeds	3		2	3
E	fry bread-crumbs with butter	4		1	∞
F	mix apples and bread-crumbs	8	A, E	0	∞

Problem 1 (2 point) Sort the tasks in topological order (either as graph, or as a list) and find the earliest starting time U_i of each task i respecting time windows. Show the recursion you are using, and report the makespan T . ■

Problem 2 (3 point) Knowing the makespan T , find the latest starting time V_i of each task i , respecting the time windows. Show the recursion you are using. ■

Problem 3 (1 point) Indicate the critical path, and describe how you found it. ■

Assignment problem

In the *Assignment problem* we are asked to assign a number of resources A_i to an equal number of activities B_j , so as to minimize total cost of allocation. Each resource should be assigned to exactly one activity, and each activity should have exactly one resource assigned.

The below cost matrix $C = \{c_{ij}\}$ says how much it costs to assign resource A_i to activity B_j .

	B_1	B_2	B_3
A_1	2	6	1
A_2	3	1	4
A_3	9	3	2

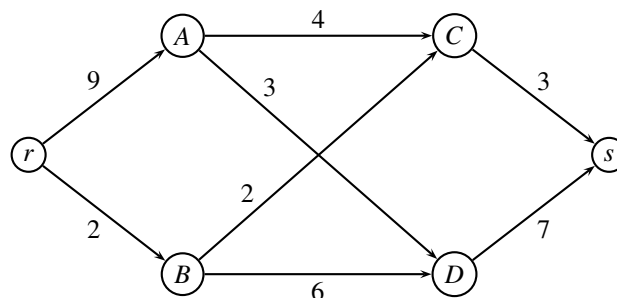
Problem 4 (2 point) Formulate the assignment problem as a transportation problem. ■

Problem 5 (1 point) Find a greedy solution for the transformed problem: Consider the rows in the order A_1, A_2, A_3 , always choosing the cheapest option for transporting. ■

Problem 6 (4 point) Solve the problem for the above matrix C , using network simplex. **Hint:** If you have too few edges in basis, add some additional edges from A_3 . ■

Maximum flow problem

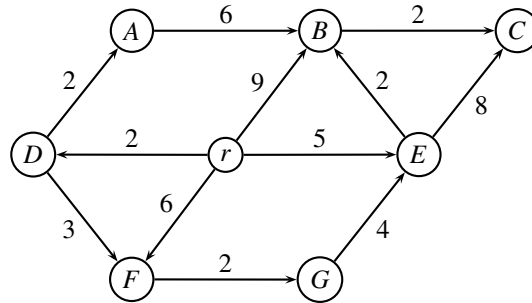
Consider the following maximum flow problem where the capacity u_{ij} is indicated on edge $(i, j) \in E$.



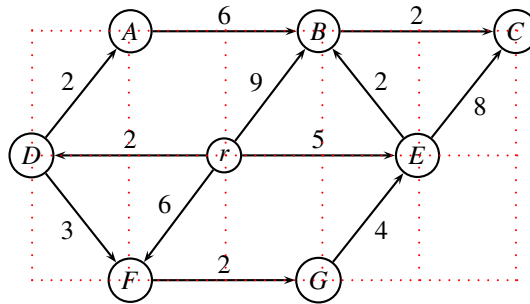
Problem 7 (5 point) Solve the problem using the *preflow-push* algorithm (also called *push-relabel*). You can use the figures on the last page, and enclose them to the solution. Remember to write what you are doing. ■

Problem 8 (2 point) Find a minimum cut and the corresponding capacity of the cut. (Explain how you find the cut). ■

A* algorithm for shortest path



Problem 9 (3 point) Consider the above directed graph with r as root, and edge weights w_{ij} . Find the shortest distance from r to all nodes using Dijkstra's algorithm. Give sufficient details to follow your calculations. ■



If we only are interested in solving the shortest distance to a specific terminal node $t = C$, we can use the A* algorithm which makes use of heuristic distances to guide the search.

The Manhattan distance between two nodes (x_1, y_1) and (x_2, y_2) is measured as $h = |x_1 - x_2| + |y_1 - y_2|$. For instance, the Manhattan distance from r to C is $h = 3 + 1 = 4$, while the Manhattan distance from F to C is $h = 4 + 2 = 6$.

Problem 10 (1 point) Calculate the Manhattan distance h_i from node i to terminal $t = C$ for all nodes. ■

The following A* algorithm finds the shortest path from r to t . It is identical to Dijkstra's algorithm, except that in every iteration we select the node i with lowest value of $y_i + h_i$. (The normal Dijkstra algorithm selects the node with lowest value of y_i). Furthermore, the algorithm stops when node t is added to T (i.e. marked "red"). All changes are written in blue.

```

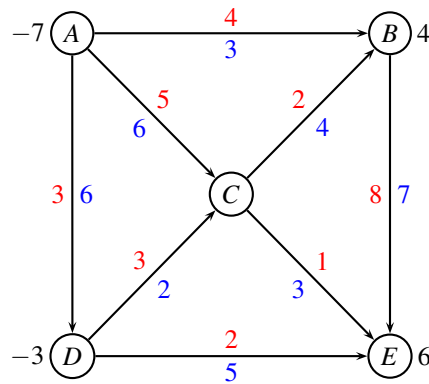
1  $T \leftarrow \emptyset$ ;
2  $y_r \leftarrow 0$ ;  $y_i \leftarrow \infty$  for all other  $i$ ;
3 while  $V \setminus T \neq \emptyset$  do
4   select an  $i \in V \setminus T$  minimizing  $y_i + h_i$ ;
5   for  $\{(i, j) \in E : j \notin T\}$  do
6     if  $y_i + w_{ij} < y_j$  then
7        $y_j \leftarrow y_i + w_{ij}$ ;
8    $T \leftarrow T \cup \{i\}$ ;
9   if  $i = t$  then
10    stop

```

Problem 11 (2 point) Use the above algorithm to find the shortest distance from r to $t = C$. Give sufficient details to follow your calculations. ■

Minimum cost flow problem

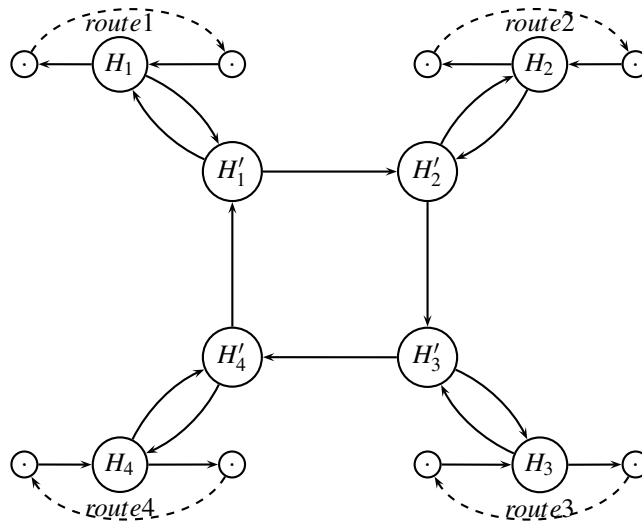
Consider the following minimum cost flow problem costs w_{ij} are indicated with **red**, capacities u_{ij} are indicated with **blue**, and demands d_i are indicated in black.



Problem 12 (2 point) Find an initial solution by solving a maximum flow problem. ■

Problem 13 (5 point) Solve the problem to optimality. ■

Modeling a hub



We study how the hub port H in a liner shipping network, HUBLINE, can be formulated in graph terms as part of a multi-commodity flow network. We have four routes arriving to the hub as follows:

route	1	2	3	4
weekday	Monday	Tuesday	Friday	Sunday

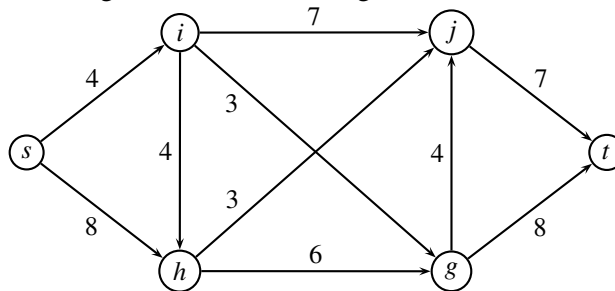
We assume that each vessel is only one day in the port, and it is assumed that all loading and unloading is done during this day.

Crane capacity: The crane capacity is $u = 5$ for unloading and $u = 4$ for loading a vessel. *Transshipment cost:* The cost of loading/unloading is 6 euro per container. *Transshipment time:* The time for transferring a container from one vessel to another can be calculated from the time table. *Storage capacity:* The port has a limited storage capacity of $\bar{u} = 9$ containers on shore. *Storage cost:* The shipping company must pay $\bar{c} = 2$ euro for each container each night it is stored in the port.

Problem 14 (3 point) Indicate the **costs**, **capacities**, **times** on the above graph. ■

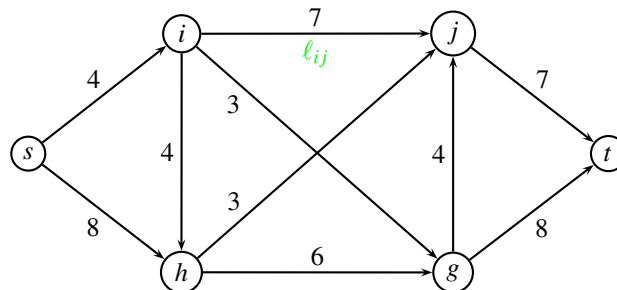
Maximum flow with lower bound on edges

Given a directed graph $G = (V, E, u)$, root/sink (s, t) and a target for the flow k . We assume that all capacities u are positive integers. All the following models should be solvable as *maximum flow problems*.



Problem 15 (1 point) How would you check whether a feasible solution exists with flow $|f| = k$? (You can use the above graph as an example, but try to argue generally).

Hint: Formulate the problem as a circulation problem. ■



Problem 16 (2 point) Now, assume that there is a lower bound ℓ_{ij} on an edge (i, j) . How would you check whether a feasible solution exists with flow $|f| = k$, respecting the lower bound? (You can use the above graph as an example, but try to argue generally. The method should also work if you have multiple edges with lower bound).

Hint: Formulate the problem as a circulation problem. ■

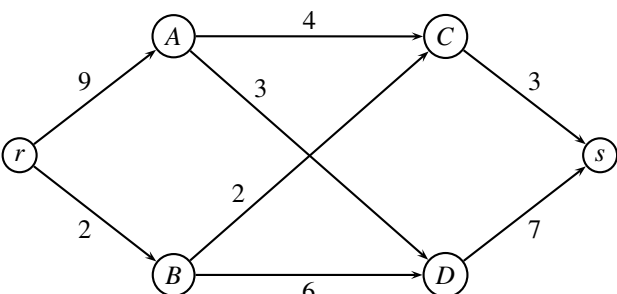
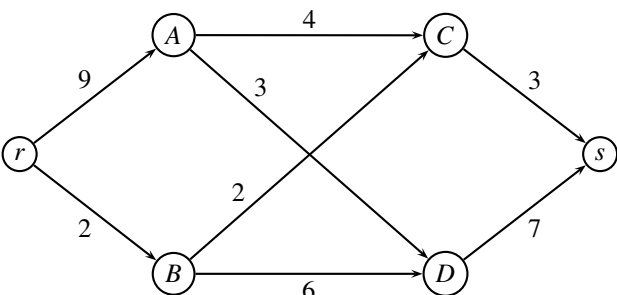
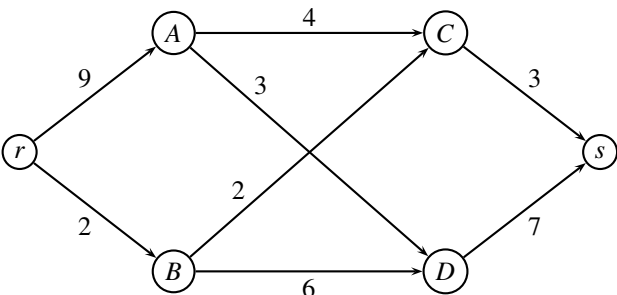
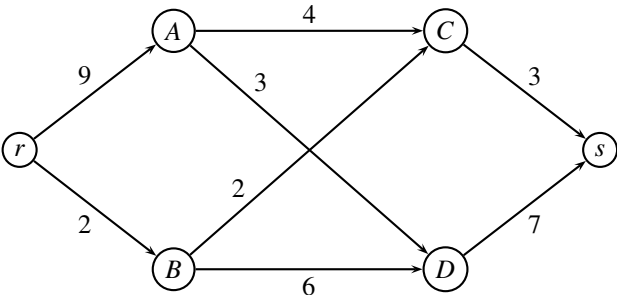
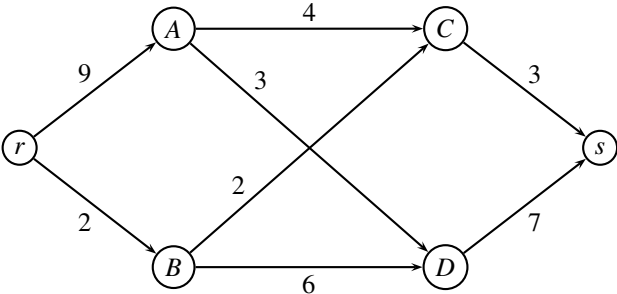
Problem 17 (2 point) Finally, consider the same problem as in previous question, but assume that k is not fixed. Assume that you would like to find a maximum flow, i.e. maximize k , How would you solve this problem?

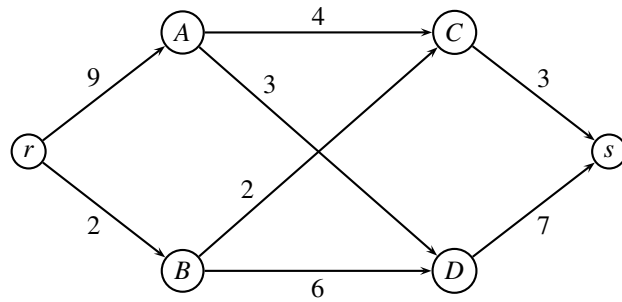
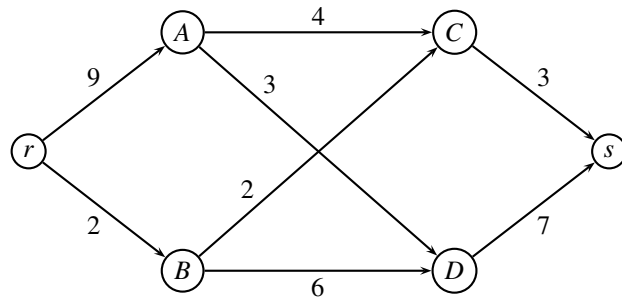
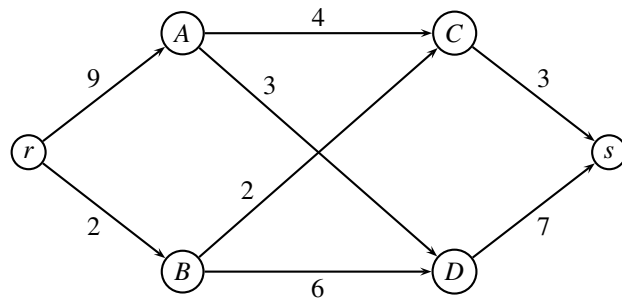
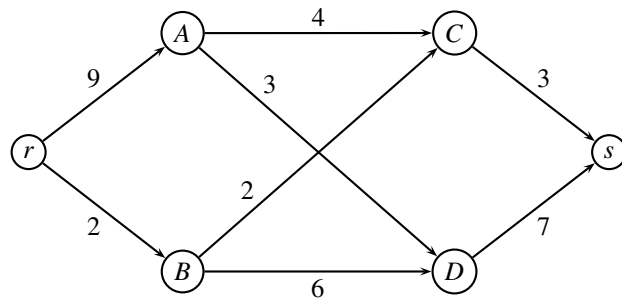
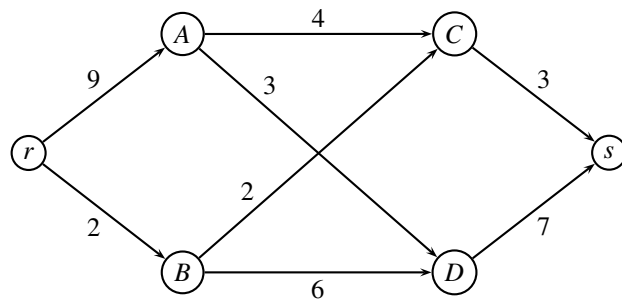
Hint: You may solve several maximum-flow problems. But try to use as few as possible. ■

_____ end of assignment _____

Maximum flow problem

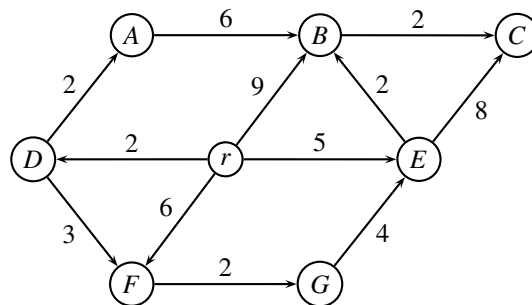
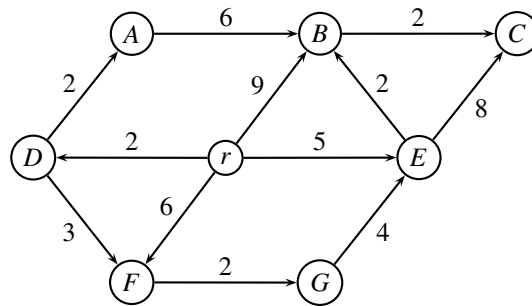
Study number:





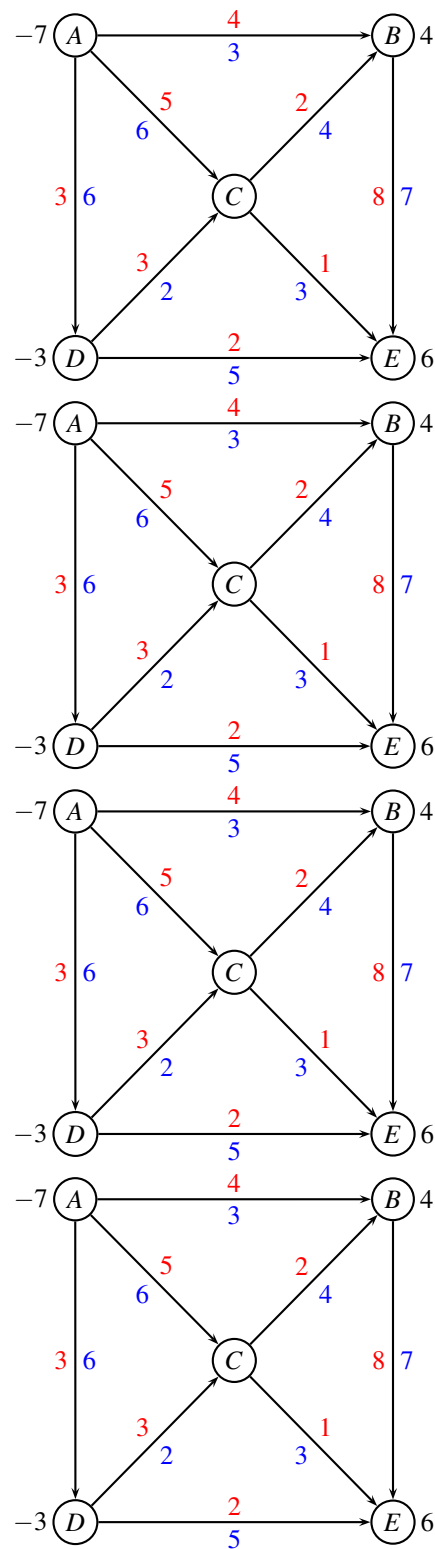
A^* algorithm for shortest path

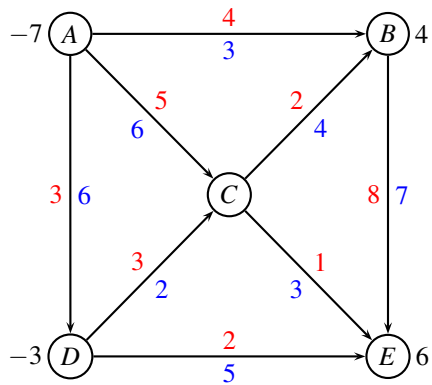
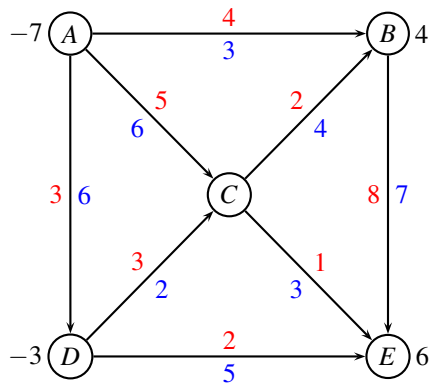
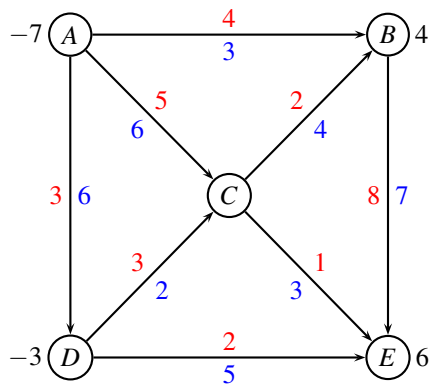
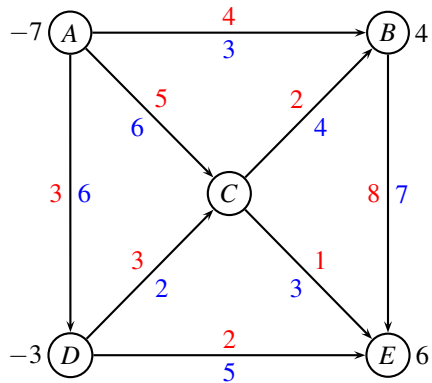
Study number:



Minimum cost flow problem

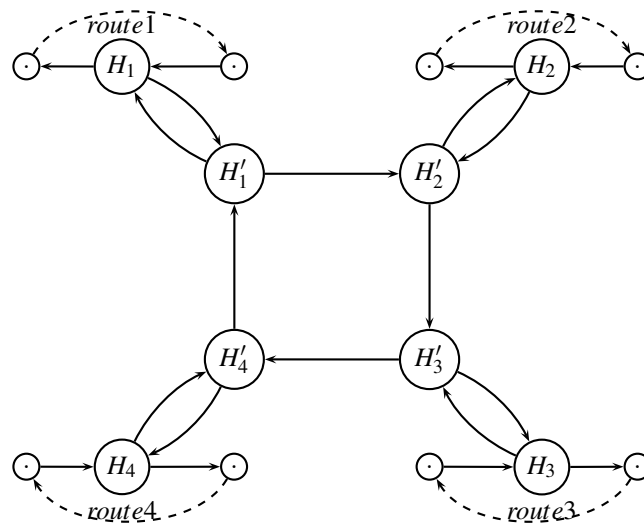
Study number:





Modeling a hub

Study number:



Maximum flow with lower bound on edges

Study number:

